

MeetFlow

Phase 1 Implementation Plan

One-to-One P2P Video Baseline

Detailed Task Division & Startup Guide for a 2-Member Team

MeetFlow Engineering Team

June 23, 2026

Contents

1	Executive Summary	3
2	Team Roles & Responsibility Matrix	3
2.1	Engineer A: Backend & Infrastructure	3
2.2	Engineer B: Frontend & WebRTC Client	3
2.3	Collaborative Milestones	4
3	Step-by-Step Startup Guide	5
3.1	Step 1: Preamble & Agreement on the Signaling Protocol (Both Engineers) . . .	5
3.2	Step 2: Backend Core Setup (Engineer A)	5
3.3	Step 3: Frontend Client Setup (Engineer B)	5
4	Detailed Task Breakdown & Timeline	6
5	Signaling Server Interface Contract	7
5.1	Client Emit: Join Room	7
5.2	Server Broadcast: User Joined	7
5.3	Client Emit: SDP Negotiation (Offer / Answer)	7
5.4	Client Emit: ICE Candidate	7
6	Testing, Diagnostics & Debugging Tips	8
6.1	Recommended WebRTC Debugging Tools	8
6.2	Common Gotchas & Solutions	8

1 Executive Summary

This document details the development roadmap and role assignment for executing **Phase 1 (One-to-One Baseline)** of the MeetFlow project.

The primary objectives of Phase 1 are:

- Establish a robust Client-Server Socket.IO signaling connection.
- Implement dynamic room creation and joining APIs.
- Set up the WebRTC `RTCPeerConnection` lifecycle to negotiate and stream peer-to-peer audio and video directly between two browsers.

This guide divides tasks for a **two-member engineering team** (referred to as **Engineer A: Backend & Infrastructure** and **Engineer B: Frontend & WebRTC Client**).

2 Team Roles & Responsibility Matrix

To complete Phase 1 efficiently, development is split into parallel paths, converging during integration and testing.

2.1 Engineer A: Backend & Infrastructure

Engineer A is responsible for the foundation server, room management logic, and the WebSocket signaling gateway.

- **Tech Stack Focus:** Node.js, Express, Socket.IO, CORS configuration.
- **Key Tasks:**
 1. Set up the Node.js development server with environment configurations.
 2. Create HTTP endpoints to generate and validate unique Room IDs.
 3. Implement the Socket.IO server to orchestrate signaling events (`join-room`, relaying `offer`, `answer`, and `ice-candidate`).
 4. Manage transient room states (e.g., tracking socket IDs of participants per room).

2.2 Engineer B: Frontend & WebRTC Client

Engineer B is responsible for the user interface, media device acquisition, socket connection client, and WebRTC peer connection management.

- **Tech Stack Focus:** React, Vite, Socket.IO-Client, WebRTC API.
- **Key Tasks:**
 1. Bootstrap the React + Vite application and set up basic routing/views.
 2. Build the UI forms to create a new meeting room or join an existing one.
 3. Implement local camera and microphone stream capture via `navigator.mediaDevices.getUserMedia`.
 4. Orchestrate the `RTCPeerConnection` state machine (handling track additions, local/remote description configuration, and ICE candidates).
 5. Design the media stage grid displaying the local and remote HTML video elements.

2.3 Collaborative Milestones

Both engineers must pair-program on the following critical integration steps:

- ****Signaling Contract Definition:**** Agreeing on exact Socket.IO event names and JSON payload schemas.
- ****End-to-End P2P Negotiation:**** Debugging the SDP offer/answer handshakes and ICE candidate exchange.
- ****Local Network Testing:**** Conducting multi-device tests (e.g., phone to laptop, or two browser tabs) on the same local area network (LAN).

3 Step-by-Step Startup Guide

3.1 Step 1: Preamble & Agreement on the Signaling Protocol (Both Engineers)

Before writing any code, both engineers must finalize the signaling protocol contract. The standard exchange pattern is diagrammed below:

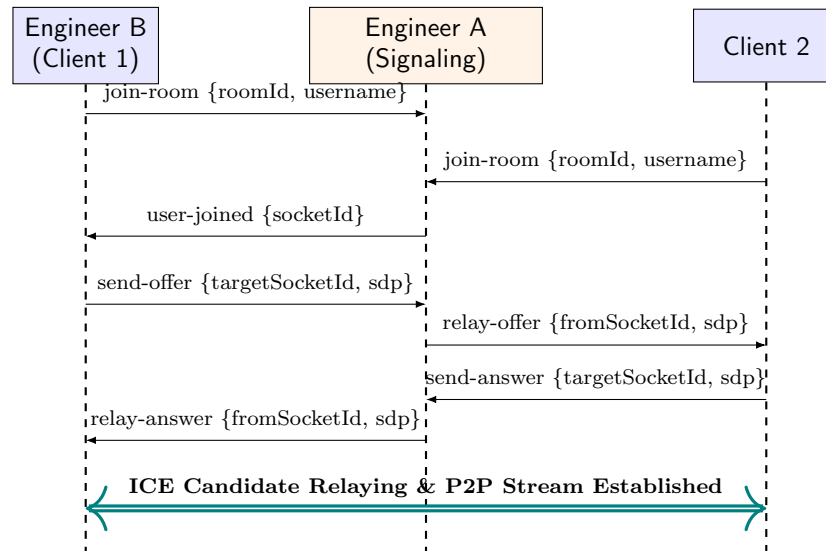


Figure 1: Phase 1 Signaling and Connection Protocol

3.2 Step 2: Backend Core Setup (Engineer A)

Engineer A initiates the workspace backend folder structure:

```

1 mkdir server
2 cd server
3 npm init -y
4 npm install express socket.io cors dotenv

```

Configure an `index.js` that boots an HTTP server listening on port 5000 and binds Socket.IO with CORS settings allowing traffic from the frontend development port (typically `http://localhost:5173`).

3.3 Step 3: Frontend Client Setup (Engineer B)

Engineer B bootstraps the client folder layout in parallel:

```

1 npm create vite@latest client -- --template react
2 cd client
3 npm install socket.io-client react-router-dom lucide-react

```

Design the routing layout using React Router:

- `/` → Landing Screen (Join Room form / Create Room button).
- `/room/:roomId` → Video Conference Room.

4 Detailed Task Breakdown & Timeline

Day	Assignee	Task Description
Day 1	Eng A	Setup basic Express app, configure environmental variables, and build a REST API (<code>POST /api/rooms</code>) returning a unique 9-digit hyphenated Room ID (<code>XXX-XXX-XXX</code>).
	Eng B	Create frontend boilerplate with routing. Design the landing page containing a text input for Room ID, user nickname, and buttons.
Day 2	Eng A	Initialize Socket.IO connection handlers. Write room state management to prevent more than two users from joining the same room (One-to-One boundary).
	Eng B	Add camera and microphone selection logic. Render the local camera track onto a <code><video></code> component with audio muted locally to prevent feedback.
Day 3	Eng A	Complete signaling event listeners (<code>offer</code> , <code>answer</code> , <code>ice-candidate</code>) that target a specific peer socket.
	Eng B	Build the Socket manager in React using React contexts or custom hooks. Verify connection states, connect client sockets, and join the correct room.
Day 4	Both	**Integration Day 1** : Link frontend client sockets to the backend. Verify logging on server when a user enters/leaves, and check user count limits.
Day 5	Eng B	Write the WebRTC core handshake: Instantiate <code>RTCPeerConnection</code> , attach media tracks, capture generated local ICE candidates, and trigger SDP offer generation upon receiving <code>user-joined</code> .
	Eng A	Assist with real-time logging, testing websocket packet integrity, and configuring fallback STUN servers.
Day 6	Both	**Integration Day 2** : Verify the complete WebRTC handshake. Configure the remote stream component, bind incoming tracks to the <code>remoteVideo</code> HTML element, and test audio/video flow.
Day 7	Both	Perform local loopback tests, verify NAT traversal on different browser engines (Chrome vs Firefox/Safari), and package codebase for Phase 2.

Table 1: Phase 1 Action Schedule

5 Signaling Server Interface Contract

To prevent runtime configuration disparities, the Socket.IO communication must adhere to these exact structures:

5.1 Client Emit: Join Room

Emitted by the client immediately upon mounting the room page.

```
1 {  
2   "roomId": "abc-123-xyz",  
3   "username": "EngineerB"  
4 }
```

5.2 Server Broadcast: User Joined

Broadcasted by the server to the existing socket in the room when a second peer joins.

```
1 {  
2   "socketId": "socket_id_of_new_peer",  
3   "username": "EngineerB"  
4 }
```

5.3 Client Emit: SDP Negotiation (Offer / Answer)

Clients wrap descriptions in these formats before sending them to the signaling relay:

```
1 {  
2   "targetSocketId": "socket_id_of_recipient",  
3   "sdp": {  
4     "type": "offer", // or "answer"  
5     "sdp": "v=0\r\no=- 52362354256..."  
6   }  
7 }
```

5.4 Client Emit: ICE Candidate

Clients must package network paths immediately upon discovery:

```
1 {  
2   "targetSocketId": "socket_id_of_recipient",  
3   "candidate": {  
4     "candidate": "candidate:842163049 1 udp 1686052607 192.168.1.100...",  
5     "sdpMid": "0",  
6     "sdpMLineIndex": 0  
7   }  
8 }
```

6 Testing, Diagnostics & Debugging Tips

To resolve WebRTC issues quickly during development, follow these standard diagnostic procedures:

6.1 Recommended WebRTC Debugging Tools

- **Chrome Internal Page:** Access `chrome://webrtc-internals/` in your browser tab during an active session. It provides real-time graphs showing bytes sent/received, connection states, and ICE selection processes.
- **Firefox WebRTC Tooling:** Navigate to `about:webrtc` for connection logs.

6.2 Common Gotchas & Solutions

1. **No Audio/Video Flowing:** Check if the browser block settings are active. WebRTC requires secure origins. Use `http://localhost` for local testing; if testing over a local network, configure the backend and frontend servers on HTTPS, or bypass browser policies in Chrome via: `chrome://flags/#unsafely-treat-insecure-origin-as-secure`.
2. **ICE Connection Fails:** Ensure you use public STUN servers in your client setup. A free, public server like `stun:stun.l.google.com:19302` must be supplied in your `RTCCConfiguration` object:

```
1 const peerConnection = new RTCPeerConnection({  
2   iceServers: [  
3     { urls: 'stun:stun.l.google.com:19302' }  
4   ]  
5 });
```

3. **Local Audio Feedback:** Ensure the local HTML `<video>` element has the `muted` attribute enabled. Do not mute the remote video stream.